

6교시: Week 1 로컬 PostgreSQL 정리와 DB 컨테이너 준비

수업 목표

- Week 1에서 설치했던 로컬 PostgreSQL이 Docker DB 실습과 충돌할 수 있음을 이해한다.
- 기존 PostgreSQL을 삭제, 중지, 또는 보류 중 하나로 안전하게 처리하고 확인 지점을 남긴다.
- PostgreSQL 공식 Docker image 문서를 읽고 POSTGRES_PASSWORD, tag, volume, port의 의미를 확인한다.

강의 전개

이 교시는 PostgreSQL을 지우는 수업이 아니라, 데이터와 port를 안전하게 다루는 수업이다. Week 1에서 로컬 PostgreSQL을 설치했다면 host port 5432를 이미 사용 중일 수 있다. Docker PostgreSQL container도 내부에서는 기본적으로 5432를 사용한다. 이 사실을 모르면 "Docker가 안 된다"가 아니라 host port 충돌인 상황을 잘못 진단하게 된다.

먼저 현재 상태를 확인한다. `psql --version`은 client가 있는지 보여줄 수 있지만 server가 실행 중이라는 뜻은 아니다. `pg_isready`, `lsof`, `ss` 같은 확인을 통해 실제 listener가 있는지 봐야 한다. 학생에게 "설치 여부"와 "실행 여부"와 "port 점유 여부"를 분리해서 확인하게 한다.

그 다음 선택지를 나눈다. 삭제, 중지, 보류는 모두 가능한 선택이다. 수업용 데이터만 있고 확실히 필요 없다면 삭제할 수 있다. 데이터는 보존하고 port만 비우고 싶다면 중지가 맞다. 회사 장비, 개인 프로젝트 DB, 권한 부족, 백업 불확실성이 있으면 보류하고 Docker host port를 15432, 15433처럼 다르게 잡으면 된다. 무조건 삭제를 정답으로 두면 데이터 안전 원칙을 해친다.

PostgreSQL 공식 image 문서는 여기서 처음 깊게 읽는다. POSTGRES_PASSWORD가 왜 필요한지, POSTGRES_USER와 POSTGRES_DB의 기본값이 무엇인지, tag가 major version을 어떻게 고정하는지, volume path가 version별로 왜 조심스러운지 확인한다. 특히 PostgreSQL 16과 18을 같은 volume에 물리는 실수를 막아야 한다.

마지막에는 다음 교시의 mental model을 만든다. container 내부 port는 둘 다 5432여도 된다. host가 외부에서 받는 port만 15432, 15433으로 다르다면 두 DB를 동시에 띄울 수 있다. 이 차이를 이해해야 Day 1의 핵심 실험인 "같은 DB port를 가진 서로 다른 version 병렬 실행"이 의미를 가진다.

로컬 PostgreSQL 상태 확인

Week 1에서 PostgreSQL을 로컬에 설치했다면 기본 port 5432를 이미 사용 중일 수 있다. Docker container 안의 PostgreSQL도 기본적으로 container port 5432에서 listen한다. host port를 같은 5432로 publish하려고 하면 기존 로컬 PostgreSQL 또는 다른 container와 충돌할 수 있다.

먼저 현재 상태를 확인한다.

```
psql --version
pg_isready
lsof -i :5432
```

Linux에서 lsof가 없으면 아래 명령을 사용한다.

```
ss -ltnp | grep ':5432'
```

확인 표

증상	의미	다음 행동
psql --version이 나온다	client 또는 server package가 설치되어 있을 수 있음	server 실행 여부를 따로 확인
pg_isready가 accepting connections	local PostgreSQL server가 실행 중	중지/삭제/보류 선택
:5432 listener가 보인다	host port 5432가 사용 중	Docker host port를 바꾸거나 기존 DB 중지
아무 것도 보이지 않는다	local DB가 없거나 실행 중이 아님	Docker DB 실습 진행 가능

삭제/중지/보류 경로 선택

삭제는 되돌리기 어렵다. 개인 장비에 중요한 데이터가 있으면 삭제하지 말고 중지하거나 보류한다. 수업의 핵심은 "무조건 삭제"가 아니라 Docker container로 DB version과 port를 격리해 실행할 수 있음을 확인하는 것이다.

선택	언제 선택하는가	확인 지점
삭제	Week 1 실습용이고 데이터가 필요 없음	삭제 명령, psql --version 또는 service 확인 결과
중지	데이터는 보존하되 오늘 port 충돌을 피하고 싶음	service stop 명령, pg_isready 실패 또는 port free
보류	삭제/중지가 위험하거나 권한이 없음	이유, 현재 port, Docker에서 다른 host port 사용 계획

OS별 정리 절차

macOS Homebrew 예시

설치 방식이 Homebrew였던 경우에만 사용한다.

```
brew services list | grep postgresql
brew services stop postgresql@16
```

완전히 삭제하기로 결정했고 데이터가 필요 없을 때만 실행한다.

```
brew uninstall postgresql@16
```

Homebrew가 아닌 installer로 설치했다면 공식/설치 방식별 문서를 확인하고 삭제한다. 데이터 디렉터리는 자동 삭제되지 않을 수 있으므로 무작정 지우지 않는다.

Linux Ubuntu 예시

패키지 설치 상태와 service 상태를 확인한다.

```
systemctl status postgresql --no-pager  
sudo systemctl stop postgresql
```

완전히 삭제하기로 결정했고 데이터가 필요 없을 때만 실행한다.

```
sudo apt remove postgresql postgresql-*
```

데이터까지 삭제하는 purge/remove 작업은 수업 중 강제하지 않는다. 기존 데이터가 있으면 먼저 백업 여부와 삭제 범위를 확인한다.

공식 postgres image 읽기

PostgreSQL 공식 image는 Docker Official Images의 postgres 문서를 기준으로 읽는다. 수업에서는 Docker Hub UI보다 문서화된 README를 우선한다.

핵심 기준:

- 실행 예시는 `docker run --name some-postgres -e POSTGRES_PASSWORD=... -d postgres` 형태다.
- `POSTGRES_PASSWORD`는 기본 초기화에 필요한 필수 환경변수다.
- `POSTGRES_USER`, `POSTGRES_DB`는 선택값이고, 지정하지 않으면 기본 user는 postgres다.
- PostgreSQL 18 이상은 PGDATA 기본 경로가 version specific 경로로 바뀌었으므로 volume 경로를 무심코 섞지 않는다.

공식 image 읽기 note

항목	오늘 사용할 값
Image tag 1	postgres:16
Image tag 2	postgres:18
Container port	5432
Host port for 16	15432
Host port for 18	15433
User	postgres
Password	postgres 또는 수업용 임시 비밀번호
Volume 16	paperclip-pg16-data
Volume 18	paperclip-pg18-data

Docker DB 실행 준비

다음 교시에는 두 PostgreSQL version을 동시에 띄운다. container 내부 port는 둘 다 5432지만 host port를 다르게 잡으면 동시에 접속할 수 있다.

```
host localhost:15432 -> postgres:16 container port 5432
host localhost:15433 -> postgres:18 container port 5432
```

Local PostgreSQL cleanup 주의점

- 로컬 PostgreSQL을 삭제하기 전에 기존 데이터가 필요한지 먼저 확인한다. 수업 실습 때문에 개인 DB를 지우면 복구가 어렵다.
- 삭제가 위험하면 중지 또는 보류를 선택하고, Docker PostgreSQL은 다른 host port를 사용한다.
- 5432가 이미 사용 중이면 Docker 문제가 아니라 host port 충돌일 수 있다. 먼저 어떤 process가 port를 쓰는지 확인한다.
- PostgreSQL 16과 18을 같은 volume에 연결하지 않는다. major version이 다른 DB data directory를 섞으면 초기화 오류나 데이터 손상이 날 수 있다.

공식 근거 링크

- PostgreSQL official image README: <https://github.com/docker-library/docs/blob/master/postgres/README.md>
- Docker Docs: Publishing and exposing ports, <https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/>
- PostgreSQL Docs: Server Applications, <https://www.postgresql.org/docs/current/app-postgres.html>

다음 연결

다음 교시는 postgres:16과 postgres:18을 실제로 실행하고, 같은 container port 5432를 서로 다른 host port로 연결하면 동시에 접근 가능한지 검증한다.